A Random Activation Push Changes a Code Model's Algorithm Where Temperature Does Not

Abstract

Sampling temperature is often treated as the creativity control of a language model. We ask whether it changes which algorithm a code model implements, rather than how the model writes the same algorithm. On shared random streams, we compare temperature in Qwen3-8B with a random push of the residual-stream states at the prompt tokens that specify the task, and we label every program by its runtime behaviour on hidden tests. The push changed the algorithm in more seeds than temperature did, by a paired difference of 0.545 per seed (95% CI 0.41 to 0.67) pooled over shortest path and k-th smallest. This contrast was preregistered and held in a single experiment whose overall preregistered verdict is partial, because the push fell short of its predicted effect on k-th smallest; it ran only in an 8-bit (W8A8) engine and is untested in bfloat16. Temperature from 0.3 to 1.3 changed the algorithm in none of 20 seeds on each of three tasks while the number of phrasings grew, a null also observed in thinking mode and in bfloat16. Prefix resampling shows why: one or two forking tokens settle the algorithm, and on these tasks the default leads there by several nats, more than this temperature range can overcome. Where the forking token is contested, a law built from its logits and committed before sampling predicted the temperature response on five of six units, including two held-out Gemma-3-4B units, though a baseline that predicts no change was nearly as accurate. No control applied to single samples produced a correct novel algorithm; with an evaluator selecting programs across generations, one bin packing search yielded eight programs that its manual read classed as a new idea, which an independent replication did not reproduce. For diversity evaluations, the results locate where randomness reaches the algorithm: at the forking token and upstream of it. Inside program search, forcing a branch at the position of highest entropy found more distinct behaviours per call than sampling at temperature 1.0 on bin packing but not on a travelling salesman task, and its advantage over forcing a branch at a random position did not replicate.

1 Introduction

Ask Qwen3-8B [45] to implement a path search on a grid and it writes breadth-first search (hereafter BFS) in 99 of 100 programs; ask it for a weighted shortest path and it writes Dijkstra's algorithm in 100 of 100. The model knows the alternatives: when asked to list ten approaches before implementing one, Qwen3-8B names every grid algorithm in our catalogue in 100 of 100 lists.[1] It simply does not use them. This paper asks which inference-time controls change the algorithm such a model implements, as opposed to the way it writes the same algorithm, and where inside generation that choice is made.

The question matters because many uses of code models depend on drawing different solutions rather than different spellings of one solution. Choosing the best of n samples and program search with an automatic evaluator [28, 34] rely on sampling to explore the space of approaches, and creativity benchmarks measure how far it does [21, 51]. Output diversity is known to narrow after alignment, a pattern called generative monoculture [41], and code models concentrate on a few algorithms among correct programs [17, 26]. Sampling temperature is the control most often raised to counter this, and it is often described as the creativity parameter of a language model [30].

Answering the question is harder than counting distinct outputs. Two programs can differ in every line and implement the same algorithm, and a program can name one algorithm while executing another, so labels

[1] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-1/experiment-6`

must come from what the code does when run. Comparisons across temperatures drawn with independent random numbers mix the effect of temperature with sampling noise; with a default share near 1.0, a few stray samples look like a temperature effect. And knowing that an algorithm changes says nothing about where the decision was made: in the prompt representation, at some point during generation, or diffusely across the program.

Prior work leaves these pieces apart. Algorithmic diversity has been measured across sampling settings with independent samples [17], and the gap between approach-level and surface-level diversity has been documented for mathematical reasoning [16]. Forking tokens, positions where resampling changes the outcome distribution, have been found in text generation [4], and post-training shifts are known to concentrate at a few high-entropy tokens [24, 39]. No study we know of has located the token that decides which algorithm a code model writes, predicted from it what temperature will do before sampling, or measured temperature against an internal perturbation under one shared random stream.

We hold one random stream fixed per seed across conditions (common random numbers), label every program by its runtime behaviour on hidden tests, and locate forking tokens by prefix resampling. On the same streams, a random push of the task specification states of the prompt changed the algorithm Qwen3-8B writes in more seeds than temperature did, by 0.545 per seed (95% CI 0.41 to 0.67) over shortest path and k-th smallest, a preregistered contrast that held, run only in an 8-bit engine and untested in bfloat16 (Figure 1). Temperature from 0.3 to 1.3 never changed the algorithm in any of 20 seeds on three tasks where the model writes one default algorithm, while the number of distinct phrasings grew. The algorithm is settled at one or two forking tokens after a shared opening, and on the default tasks the default leads there by 4.85 to 17.04 nats, far more than temperature in the range 0.3 to 1.3 can overcome. Where the forking token is contested, a law composed from its teacher-forced logits predicts how temperature moves the mix. Our results cover one main model family, a small set of tasks, and a temperature law tested on only three held-out units, two of which passed.

Summary of contributions.

- **A task-state push changes the algorithm where temperature does not (Section 5.1).** On the same random streams, a random task-state push, that is, one random direction added to the task specification states (Section 3.3), changed the algorithm in more seeds than temperature did, by 0.545 per seed (95% CI 0.41 to 0.67), pooled over shortest path and k-th smallest (40 seeds). The contrast was preregistered and held (sign flip test, $p = 1.0 \times 10^{-5}$) in an experiment graded partial, and it ran only in an 8-bit (W8A8) engine. Temperature 0.3 to 1.3 changed the algorithm in 0 of 20 seeds on each of three tasks while phrasing clusters on the grid task rose from 3 to 13, a null that replicates in thinking mode, over 800 independent samples per task and, at the forking token, in bfloat16.

- **One or two forking tokens settle the algorithm (Section 5.2).** Given the class of the forking token, at most 0.19 bits of uncertainty about the algorithm remain; on four default tasks the fork margin (Section 3.4) is 4.85 to 17.04 nats.

- **A composed forking-token law predicts temperature's effect (Section 5.3).** Committed before sampling, it matched the observed algorithm distribution across four temperatures within a mean total variation distance of 0.014 to 0.054 on five of six units, including two held-out Gemma-3-4B units, and failed on Llama-3.1-8B's grid task (0.142). A baseline that predicts no change was close (the law was better on four of six units).

- **The task specification states and post-training set the fork (Section 5.4).** Writing the task specification states of a prompt that names the minority algorithm into a copy without that instruction, with its token positions aligned, produced the minority algorithm in 30 of 30 samples on each of three tasks.

On the three default tasks, post-training widened fork margins by 2.11 to 13.41 nats relative to the base model while changing the entropy at phrasing positions by less than 0.5 nats.

- **Other controls reweight the same repertoire (Sections 5.5 and 5.6).** Random seeds, novelty instructions, bans, activation pushes and program search moved programs among known algorithms. No audit of a control applied to single samples found a correct novel algorithm; eight bin packing programs from one program search were classed as a new idea, and a replication found none. Where a ban of five names lands in activation space depends on whether the map is built from names or from programs.

Section 2 places the work against prior studies, Section 3 describes the tasks, labels and interventions, Section 4 gives the experimental setup, Section 5 reports the results, and Sections 6 to 8 discuss them, state their limits and conclude.

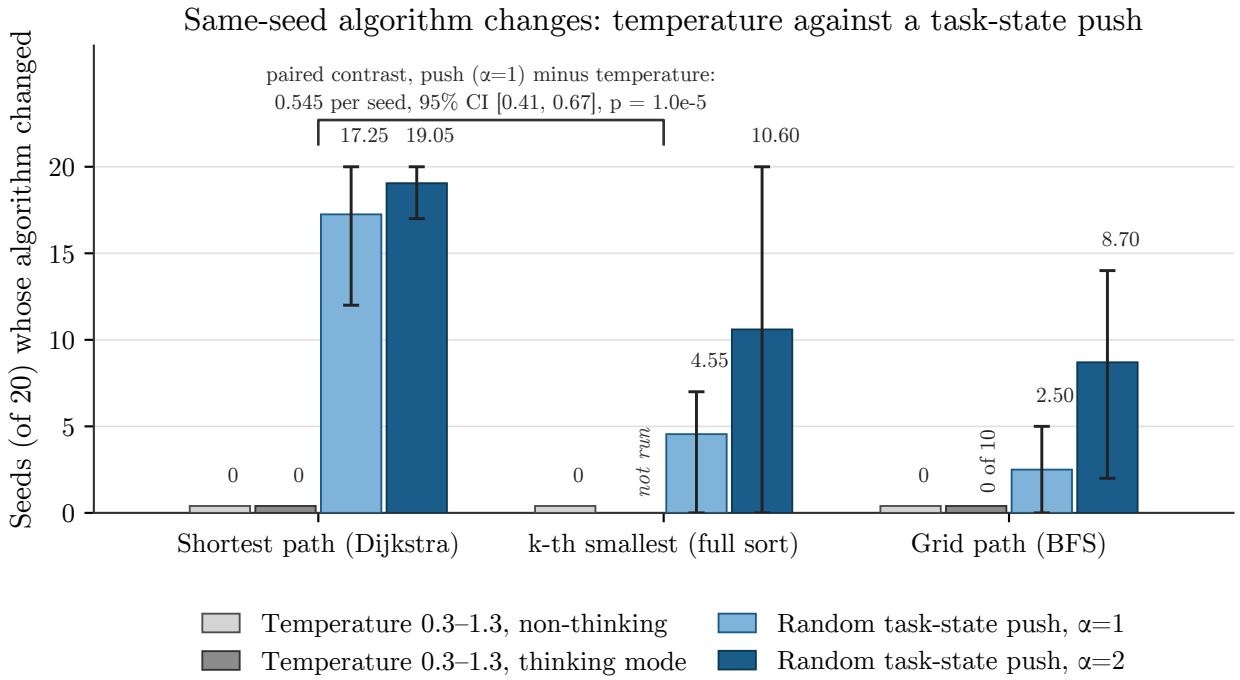


Figure 1: Seeds (of 20) whose algorithm label changed under one shared random stream per seed, for Qwen3-8B on an 8-bit (W8A8) engine; grouped by task along the horizontal axis (shortest path/Dijkstra, *k*-th smallest/full sort, grid path/BFS), with four bars per task as in the legend below the axes. Light and mid grey bars sweep temperature over $T = 0.3, 0.7, 1.0, 1.3$ with thinking off and on; a seed counts as changed if its label differs anywhere across those four temperatures, and temperature changed no seed on any task it was run on (exact 95% upper bound 0.168 per seed). Thinking-mode cells are marked where they depart from the full 20-seed design: not run for *k*-th smallest, and completed on only 10 of 20 seeds for grid path ("0 of 10"). Light and dark blue bars add a random push to the hidden state of the model at the task specification tokens at $\alpha = 1$ and $\alpha = 2$ ($T = 1.0$); each bar is the mean, over the 20 subsets of three directions drawn from 6 independently sampled push directions, of how many seeds had their label differ from the unpushed arm anywhere among that subset, with whiskers at the minimum and maximum over subsets. The bracket over shortest path and *k*-th smallest gives the preregistered paired contrast, push ($\alpha = 1$) minus temperature, pooled over those two tasks: 0.545 seeds changed per seed, 95% CI [0.41, 0.67], one sided sign flip $p = 1.0 \times 10^{-5}$. This design ran on one 8-bit engine only – bf16 does not fit the 16 GB GPU used – and was not rerun in bf16.

2 Related Work

Diversity collapse and creativity in code models. Alignment narrows output diversity relative to the variety available for a task [14, 41], reduces conceptual diversity [25], and leaves base models better at randomness and creative generation than their aligned descendants [40]. The structure of the chat format alone contributes to this collapse [47]. The typicality bias learned from human preferences has been linked to mode collapse that verbalized sampling partly undoes [49]. In code, Lee et al. [17] find that models concentrate on few algorithms among correct solutions, and Narula et al. [26] show that subtle prompt cues steer which algorithm is chosen. Denial prompting, which forbids the technique of the previous solution, is used to measure creativity in code generation [21]. Benchmarks of humanlike diversity [51] and the distinction between approach-level and surface-level diversity in mathematical reasoning [16] motivate counting ideas separately from phrasings. We adopt that separation for code, with ideas defined by runtime behaviour, and we measure it under coupled randomness so that the effect of a control on a single sample can be read directly. Our behavioural design follows an open repository of creativity experiments with language models,[2] which asked a closed model for weighted shortest path programs under context seeds, creativity instructions and cumulative bans.

Temperature and decoding. Truncation samplers such as nucleus sampling [10] and min-p sampling [27] shape the token distribution at each step, and adaptive schemes raise temperature only where the next token is hard to predict [50, 53]. Peeperkorn et al. [30] find that temperature correlates only weakly with novelty in story generation. In code, Lee et al. [17] report that temperature above 1 raises algorithmic diversity, and Sohail and Ali [36] observe more distinct operations in generated sorting algorithms as temperature rises from 0.75 to 1.1. Intent factored generation and plan search diversify a sampled intent or plan before writing [1, 38], random embeddings injected into the input diversify reasoning paths [13], and the level at which randomness enters a generation pipeline changes how diverse its outputs are [48]. In diffusion language models, an idealized model of fork tokens motivates tempering the choice of which position to unmask, rather than the token distribution, to keep entropy at the forks [29]. Coupling counterfactual generations through shared Gumbel noise [32] is the closest precedent for our common-random-numbers design. Our contribution is to explain why temperature does or does not reach the algorithm, and to predict the size of its effect from one token's logits before sampling.

Where an output is decided. Forking Paths analysis resamples continuations from prefixes and finds tokens after which the outcome distribution changes abruptly [4]. Thought anchors identify the reasoning sentences whose resampling matters most [5]. A small set of high-entropy minority tokens carries most of the effect of reinforcement learning on reasoning [39]. Post-training shifts concentrate at a few tokens [24], aligned models mostly reproduce the base model's top token at ordinary positions [19], and learned behaviours can be enforced in the first few response tokens [31]. Self-improvement and post-training have been described as sharpening probability onto outputs the base model already ranks highly [11]. We extend this line to the algorithm a program implements: we locate its forking token blind, compose a temperature law over it, and measure how far post-training moved that token relative to ordinary positions.

Interventions on internal states and program search. Activation steering adds a direction to the residual stream [2, 33], distributed alignment search fits subspaces in which interchange interventions reproduce a counterfactual [7, 42], and such subspace edits can act through pathways the model does not use naturally [22]. Activation steering has also been turned into an exploration method that pushes the hidden states of concurrent

[2] `https://github.com/DrejcPesjak/llm-creativity`

generations apart, which raises the diversity of reasoning paths and of generated test cases [12, 52]. Representation geometry is studied with geodesic distances and Isomap [37] and, recently, manifold steering [43]; sparse autoencoders [18] and the tuned lens [3] read intermediate states. Program search with a language model as mutator and an automatic evaluator has found new constructions and heuristics [20, 28, 34, 46]. We use activation edits as tests of where the algorithm is set rather than as a diversity method, and we audit every correct program outside each task's catalogue for novelty.

3 Method

Figure 2 summarises the method. Programs are labelled by what they do when run (Section 3.1). Ideas are counted separately from phrasings (Section 3.2). Conditions are compared on shared random streams (Section 3.3). The token that decides the algorithm is located by prefix resampling (Section 3.4). A temperature law is composed from that token's logits (Section 3.5). Activation exchanges test what sets the token (Section 3.6).

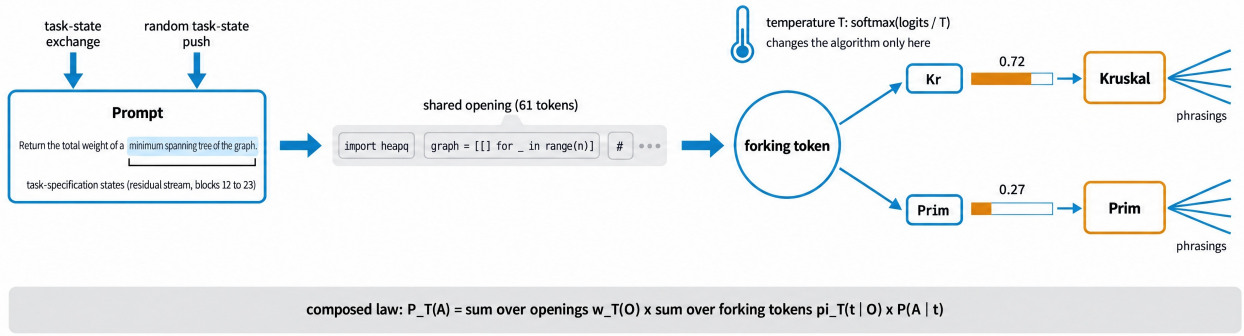


Figure 2: Schematic illustration of the account and of the measurements (not data), using one minimum spanning tree example. Left to right: a rounded 'Prompt' box contains the task text 'Return the total weight of a minimum spanning tree of the graph.', with the words that define the task, 'minimum spanning tree of the graph.', highlighted in light blue and bracketed as 'task-specification states (residual stream, blocks 12 to 23)', that is, the residual stream activations at those tokens; two downward arrows above the box, labelled 'task-state exchange' and 'random task-state push', denote interventions applied to these states. An arrow leads to the 'shared opening (61 tokens)', shown as three representative monospace code tokens ('import heapq', 'graph = [[] for _ in range(n)]', '#') followed by an ellipsis marking the rest of the shared opening. A second arrow leads to a 'forking token' node, above which a thermometer icon is labelled 'temperature T: softmax(logits / T)' with the note 'changes the algorithm only here'. From the forking token, two branches lead to the token chips 'Kr' and 'Prim'; the orange bar beside each chip shows the token's probability at the fork — 0.72 for 'Kr', leading to the 'Kruskal' outcome box, and 0.27 for 'Prim', leading to the 'Prim' outcome box, with each bar's fill proportional to its value. A fan of four thin lines labelled 'phrasings' beside each outcome box denotes that one algorithmic idea is written out in many surface forms. A grey band spanning the bottom states the composed law 'P_T(A) = sum over openings w_T(O) x sum over forking tokens pi_T(t | O) x P(A | t)'. Blue marks the pipeline from prompt to fork (boxes, arrows, the forking-token node and chips); orange marks the two realized algorithm outcomes and their fork probabilities. The 61-token opening length and the 0.72/0.27 fork split are the only numbers in the figure, drawn from teacher-forced measurements on this example; the diagram as a whole is a schematic of the proposed mechanism, not a plotted dataset.

3.1 Tasks, models and algorithm labels

The study starts from two path search tasks. In the grid task, `solve(grid, start, goal)` returns a valid path on a grid with walls; its catalogue holds BFS, depth-first search (hereafter DFS), bidirectional BFS, iterative deepening DFS, A* and unit weight Dijkstra, that is, Dijkstra's algorithm with every edge

weight set to one. The weighted shortest path task works on a directed graph, with a catalogue of Dijkstra, Bellman-Ford with its queue variant SPFA, bidirectional Dijkstra, Floyd-Warshall and Dial's algorithm. Each has 40 to 48 seeded hidden test cases with a time limit per case, and reference implementations of every catalogue algorithm that pass all tests.[3] A further 13 tasks, the *bench* below, were derived from LeetCode problems with frozen wording and catalogues of 3 to 6 algorithms each, 684 hidden test cases in total, and references for all 54 catalogue algorithms.[4] The tasks used below are k-th smallest element (default: full sort), connected components (default: union find), minimum spanning tree weight (Kruskal or Prim), trapped rain water (prefix and suffix maxima or two pointers) and valid anagram (sort and compare or counting), plus substring search and first and last position in a sorted array for Gemma. A *unit* is a task paired with one model. We call a unit a *default task* when the instruct model writes one algorithm in essentially all samples, and a *mixed unit* when its unperturbed samples already contain a second algorithm at a share of roughly 0.1 to 0.4.

The main model is Qwen3-8B [45] in non-thinking mode unless stated, with Qwen3-8B-Base for the post-training comparison. Held-out units come from Gemma-3-4B (gemma-3-4b-it, Gemma below) and Llama-3.1-8B (Llama-3.1-8B-Instruct, Llama below) [8], and Qwen3-14B and Qwen3-32B with thinking enabled enter the ban experiments. Generation used vLLM [15] or a Hugging Face engine in bfloat16 or with 8-bit floating-point linear layers, stated per experiment.

A program's *algorithm label* comes from a sandboxed runtime tracer that records queue, stack and heap operations, edge relaxations and grid reads, combined with AST rules; a blind LLM judge breaks disagreements. Labellers were gated on references and adjudicated gold sets before use. On the bench tasks, agreement between the runtime and AST labellers reached Cohen's κ 0.84 to 1.00, with every reference correctly labelled. On the path tasks the combined label agreed with the adjudicated gold label at $\kappa = 0.966$ with 22 of 22 references correct, although the stricter criterion of that gate, agreement between the runtime and AST labellers ($\kappa = 0.753$), was not met. A program is correct when it passes every hidden test.

3.2 Ideas and phrasings

We call the runtime label of a correct program its *idea*. A *phrasing cluster* is a connected component, within one idea, of programs whose AST 4-gram shingle sets, computed with normalised identifiers, have Jaccard similarity at least θ; identifiers are renamed in order of appearance, and comments, docstrings and constants are dropped. $\theta = 0.8$ is primary, with 0.7 and 0.9 as sensitivity values. We call the number of phrasing clusters within an idea its *phrasings*. Diversity over ideas is the Hill number of order 1 (Hill-1), the exponential of the Shannon entropy of the idea distribution [9], computed among correct programs and rarefied to a depth of 25 programs. The *number of ideas at depth 25* is the expected count of distinct ideas among 25 correct programs drawn from a cell.

3.3 Common random numbers

Every comparison of decoding conditions samples by inverse transform sampling with common random numbers: each seed owns one stream of uniform variates, and every condition maps the same variates through its own next-token distribution. A difference between two conditions on one seed is therefore caused by the condition, not by a different draw. For a block of conditions, the *change count* of a task is the number of seeds whose algorithm label differs anywhere across the block; we bound its rate with the Clopper-Pearson

[3] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-1/experiment-1`

[4] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-2/experiment-11`

interval [6]. Thinking-mode arms ran in vLLM with a seed set for each request, which couples conditions through shared Gumbel noise [32].

The temperature block ran each of 20 seeds per task (spread over five prompt templates) at T = 0.3, 0.7, 1.0 and 1.3 with top-p 1, and as sensitivity conditions with top-k 10 at $T = 0.7$ and with min-p 0.1 at $T = 2.0$.[5] The comparison intervention is a random activation push: activation addition with a random rather than a learned direction, akin to noise steering. We call it a *random task-state push*: one isotropic unit direction is added, during the prefill only, to the residual stream activations at the token positions that state the task (the *task specification states*) in blocks 12 to 23. Its size is set in multiples α of the displacement that a ban prompt induces at the same positions. Six fixed directions were run at $\alpha = 1$ and 2.

3.4 Locating the forking token

Programs of different algorithms often share their first tokens; we call this common prefix the *shared opening*. Following Forking Paths analysis [4], we sample base programs, resample continuations from their prefixes at increasing lengths, label every completed continuation, and detect a change point in the algorithm distribution with a preregistered Bayesian rule calibrated against permutation nulls. The change point is refined to a single token and confirmed by forcing each candidate token and sampling continuations from it. When the branch after a forking token is not pure, a second forking token is searched inside it. Below, *the fork* is short for the forking token. We define the *fork margin* as the teacher-forced gap in log probability, at the forking token, between the class of tokens that lead to the default algorithm and the best class that leads elsewhere, and report $H(A \mid \text{class})$, the entropy of the algorithm given the forking token's class.

3.5 A composed temperature law

Let O_c be the c-th shared opening that precedes a forking token, and $w_T(O_c)$ the probability of sampling it at temperature T, the product of tempered teacher-forced token probabilities along it. Let $\pi_T(t \mid O_c)$ be the tempered next-token distribution at the fork, renormalised over the top 10 logits for top-k 10, and $P(A \mid t)$ the share of algorithm A among $T = 1$ continuations forced from token t. We call the prediction

$$P_T(A) = \sum_c w_T(O_c) \Big[\sum_t \pi_T(t \mid O_c)\, G(t,A) + r_T(c)\, P(A \mid \text{rest}_c) \Big] + \Big(1 - \sum_c w_T(O_c) \Big)\, q_{\text{rest}}(A) \qquad (1)$$

the *composed law*. Here $G(t,A) = P(A \mid t)$ unless t opens a second fork, in which case G is the same mixture taken over the second fork's tokens. The term $r_T(c)$ is the tempered mass outside the top 20 tokens, and q_{rest} is the composition of samples that leave the located openings. The prediction is averaged over five prompt templates and uses only teacher-forced logits and $T = 1$ conditionals, so it can be committed before any temperature sample is drawn.[6]

Four rivals were committed with it: a flat law that keeps the $T = 1$ shares at every temperature; a rival we call share tempering, which raises the $T = 1$ shares to the power $1/T$ and renormalises; a single fork law without the second level; and the same law at the position of highest entropy instead of the located fork. Each predictor is scored by its mean absolute error (MAE), the mean over the four temperatures of the total variation distance between predicted and observed label distributions.

[5] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-4/experiment-21`

[6] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-8/experiment-43`

3.6 Probing what sets the forking token

Task-state exchange. We call the following interchange intervention a *task-state exchange*. A forced prompt (`Implement the task with ⟨minority algorithm⟩`) is paired with a *padding twin* in which the instruction is replaced by padding tokens so that all positions align. We write the task specification states of the forced prompt into the forward pass of the twin and sample on shared streams, against a random exchange matched in norm.[7]

Base against instruct. Qwen3-8B and Qwen3-8B-Base are teacher-forced on identical token ids formatted with the chat template, at the forking tokens and at 64 phrasing positions of 20 programs per task. We compare fork margins, next-token entropies and Jensen–Shannon divergences, and count how often the token of the instruct model is the top token of the base model, following the URIAL token categories [19].

3.7 Other controls and the novelty audit

To bound the result we also measured random context seeds (`seed=d32qxk4nt43` and `seed=(horse chair roof love)` prepended to the prompt) and a *semantic seed*, a paragraph that implies an algorithm without naming it. We further measured instructions from `be creative` to a 2,000-word request for a fully novel algorithm, and *bans*, instructions not to use a named algorithm, either of the default alone or of five algorithms at once (we call the latter a *ban of five names*). A *survivor* is a correct program whose algorithm is not banned. Finally, we ran an island-model program search in the manner of FunSearch [34], in which the model mutates programs and a fixed evaluator keeps the best ones across generations. Every correct program labelled outside its task's catalogue was compared with a corpus of known solutions (about 18,000 programs built from LeetCode, textbook and benchmark solutions), read by blind LLM judges gated on known controls, and read manually. Zero novel programs in n correct ones is reported with the rule-of-three bound $3/n$.

4 Experimental Setup

Preregistration. Every decisive prediction, threshold and analysis rule was written to a SHA-256 hash chain before the samples it governs were drawn, and each artifact's chain verifies end to end. The experiment with shared seeds preregistered four predictions: a temperature change count of at most 1 of 20 seeds in each of four cells (three tasks without thinking, shortest path with thinking); phrasings per idea that grow with temperature; a random push that changes at least 5 of 20 seeds on shortest path and on k-th smallest; and a positive contrast per seed between push and temperature, pooled over those two tasks, at one sided $p < 0.05$, which was the experiment's single entry in its secondary family of preregistered tests. The experiment is confirmed only if all four hold. The composed law's criterion was an MAE of at most 0.10 with $H(A \mid \text{class})$ at most 0.3 bits per unit, and passing on at least 70% of scored units including at least 3 held-out units, with at least 4 held-out units scored.

Units and samples. The temperature block used 20 seeds per task and condition for Qwen3-8B on grid, shortest path and k-th smallest, run in an 8-bit engine whose rows reproduced the same cells of an earlier run token for token (200 of 200). The composed law was developed on trapped rain water, valid anagram and minimum spanning tree weight in Qwen3-8B and tested, before any Qwen3-8B sweep sample, on held-out units from two other families: substring search and first and last position in gemma-3-4b-it, and the grid task in Llama-3.1-8B-Instruct. Each unit was swept on 100 fresh shared streams at $T = 0.3$, 0.7, 1.0 and 1.3

[7] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-7/experiment-35`

under top-p 1 and top-k 10, in bfloat16. Algorithm changes under activation pushes are not an artefact of the 8-bit engine: push directions move the algorithm about as often in bfloat16 as in the 8-bit engine (26 of 40 directions escape, by the criterion of Section 5.5, on connected components against 29 in the 8-bit engine, and 6 against 8 on shortest path).[8]

Statistics. Paired binary contrasts use the exact McNemar test [23]; the contrast per seed between push and temperature uses a sign flip test, a permutation test that flips the sign of each seed's difference, with a bootstrap interval over seed slots; and proportions use Wilson or Clopper-Pearson intervals. Every headline number was rederived from raw rows by an independent script sharing no code with the analysis, and placebo versions of each test (shuffled conditions, shuffled temperature labels, shuffled token maps) were run and rejected at about their nominal rate. Code, prompts, raw generations, labels and the hash chains of every experiment are released with the paper, and each result below links to the code that produced it.

5 Results

5.1 A task-state push changes the algorithm where temperature does not

On the same random streams, a random task-state push, that is, one random direction added to the task specification states, changed the algorithm of Qwen3-8B in more seeds than temperature did, by a paired difference of 0.545 per seed (95% bootstrap CI 0.41 to 0.67), pooled over shortest path and k-th smallest (40 seeds; Figure 1 and Table 1). For each seed, the contrast is the push's change indicator, averaged over the 20 subsets of three directions (each a union over its three directions at $\alpha = 1$ and $T = 1.0$), minus the seed's change indicator across the four temperatures. It was the experiment's single entry in its secondary family of preregistered tests, and it held, with a one sided sign flip p of 1.0×10^{-5}. A permutation of conditions within each seed never reaches the observed mean (maximum placebo mean 0.155 in 200 permutations). The experiment as a whole is graded partial: its preregistered prediction that the push changes at least 5 of 20 seeds held on shortest path (17.25) but failed on k-th smallest (4.55). The contrast was run only in the 8-bit (W8A8) engine and was not rerun in bfloat16. The push itself does change the algorithm in bfloat16 at the same absolute push norm (Section 4).

Table 1: Seeds (of 20) whose algorithm changes across the conditions of a block, Qwen3-8B, same random stream per seed, 8-bit W8A8 engine except the thinking-mode arm (vLLM). Push rows, that is, rows for the random task-state push, are the mean over the 20 subsets of three of the six fixed directions of the number of seeds whose label differs anywhere among the unpushed arm and the three directions.

Conditions on the same seeds	Shortest path (Dijkstra)	k-th smallest (full sort)	Grid path (BFS)
Temperature 0.3, 0.7, 1.0, 1.3, top-p 1	0	0	0
The same, plus top-k 10 at T 0.7	0	0	0
The same, plus min-p 0.1 at T 2.0	1	1	0
Temperature 0.3 to 1.3, thinking mode	0	not run	0 (of 10)
Random task-state push, $\alpha = 1$, T 1.0	17.25	4.55	2.50
Random task-state push, $\alpha = 2$, T 1.0	19.05	10.60	8.70

A single push direction at $\alpha = 1$ changed 7, 10, 15, 17, 3 and 8 of 20 seeds on shortest path and up to 7 of 20 on k-th smallest. The algorithms the changed programs moved to, their *landings*, were Bellman-Ford/SPFA

[8] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-6/experiment-29`

(52 of 60 programs off the default on shortest path) and quickselect (11 of 11 on k-th smallest). The push is not free: pass rates on shortest path fell from 1.00 to 0.63 at $\alpha = 1$ and 0.42 at $\alpha = 2$.

Temperature, on the same seeds, never changed the algorithm on any of the three default tasks. Across T = 0.3, 0.7, 1.0 and 1.3, all 20 seeds kept their algorithm on shortest path (Dijkstra), on k-th smallest (full sort) and on the grid task (BFS). The change count is 0 of 20 per task, with an exact 95% upper bound of 0.168 on the change rate per seed. This was a separate preregistered prediction (at most 1 of 20 seeds per cell), and it held under every reading: counting only correct programs, counting invalid outputs as a label, and adding top-k 10 as a fifth condition all leave the count at 0. Adding min-p 0.1 at $T = 2.0$ changes one seed on shortest path and one on k-th smallest. With thinking mode enabled, shortest path kept Dijkstra in 20 of 20 seeds, every final program and every first approach named in the thinking trace was Dijkstra, and all 100 traces completed within the 16k-token cap. The reduced grid arm kept BFS in 10 of 10 seeds.

Phrasings, in contrast, grow with temperature (Figure 3). On the grid task the number of phrasing clusters among 20 programs rose from 3 at $T = 0.3$ to 10, 12 and 13 at T = 0.7, 1.0 and 1.3, while the number of distinct algorithms stayed at 1 on all three tasks at every temperature. The pooled slope of log phrasings per idea on temperature is 0.70 per unit T (95% CI 0.35 to 0.93) at Jaccard threshold 0.8, and 0.64 [0.30, 0.89] and 0.81 [0.33, 0.98] at 0.7 and 0.9. Almost all of the growth comes from the grid task, while shortest path moves from 1 to 2 clusters and k-th smallest stays at 2.

The temperature null replicates outside the design with common random numbers and outside the 8-bit engine. With 100 independent samples per cell, 797 of 800 grid programs were BFS and 797 of 800 shortest path programs were Dijkstra across eight temperature and sampler cells in non-thinking mode, and 800 of 800 in thinking mode. Over the same cells, grid phrasing clusters at threshold 0.8 rose from 4 to 36 with top-k 10 and from 3 to 24 with top-p 0.95. The literal setting of thinking mode with top-k 10 under one seed per stream again changed the shortest path algorithm in 0 of 20 seeds (one sided 95% upper bound 0.139),[9] and in bfloat16, 100 samples at $T = 1.3$ from the forking position left the default unchanged on four tasks (Section 5.2). At tight temperature the model therefore writes one idea in a few phrasings, and loosening temperature multiplies the phrasings of that idea rather than adding ideas.

Which default a model writes depends on the model. Asked for a path search on a grid, Qwen3-8B writes BFS in 99 of 100 programs at $T = 1.0$ and Dijkstra in 100 of 100 on weighted shortest path. Gemma-3-4B writes DFS in 139 and BFS in 61 of 200 grid programs.[10] Llama-3.1-8B attempts BFS in 77 and DFS in 20 of 100, but its correct grid programs are all BFS.[11]

5.2 One or two forking tokens settle the algorithm

Prefix resampling places the decision at a single token after a shared opening on every unit where the instruct model writes two algorithms (Table 2, fourth column). On minimum spanning tree weight the 61-token opening is followed by the word after the comment marker `#:` `␣Kr` (probability 0.72) leads to Kruskal in every continuation, and `␣Prim` (0.27) to Prim in 97%. On trapped rain water, after `if not height: return 0`, `␣n` leads to prefix and suffix arrays and `␣left` to two pointers. On valid anagram, after the length check, `␣return` leads to sorting and `␣count` to counting. Given the class of the located forking token, the remaining entropy of the algorithm, $H(A \mid \text{class})$, is 0.050 bits [95% CI 0.014, 0.096], 0.153 and 0.189 bits on these three units, against the preregistered limit of 0.3. A blind locator with a frozen

[9] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-5/experiment-27

[10] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-6/experiment-31

[11] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-2/experiment-10, https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-3/evaluation-2

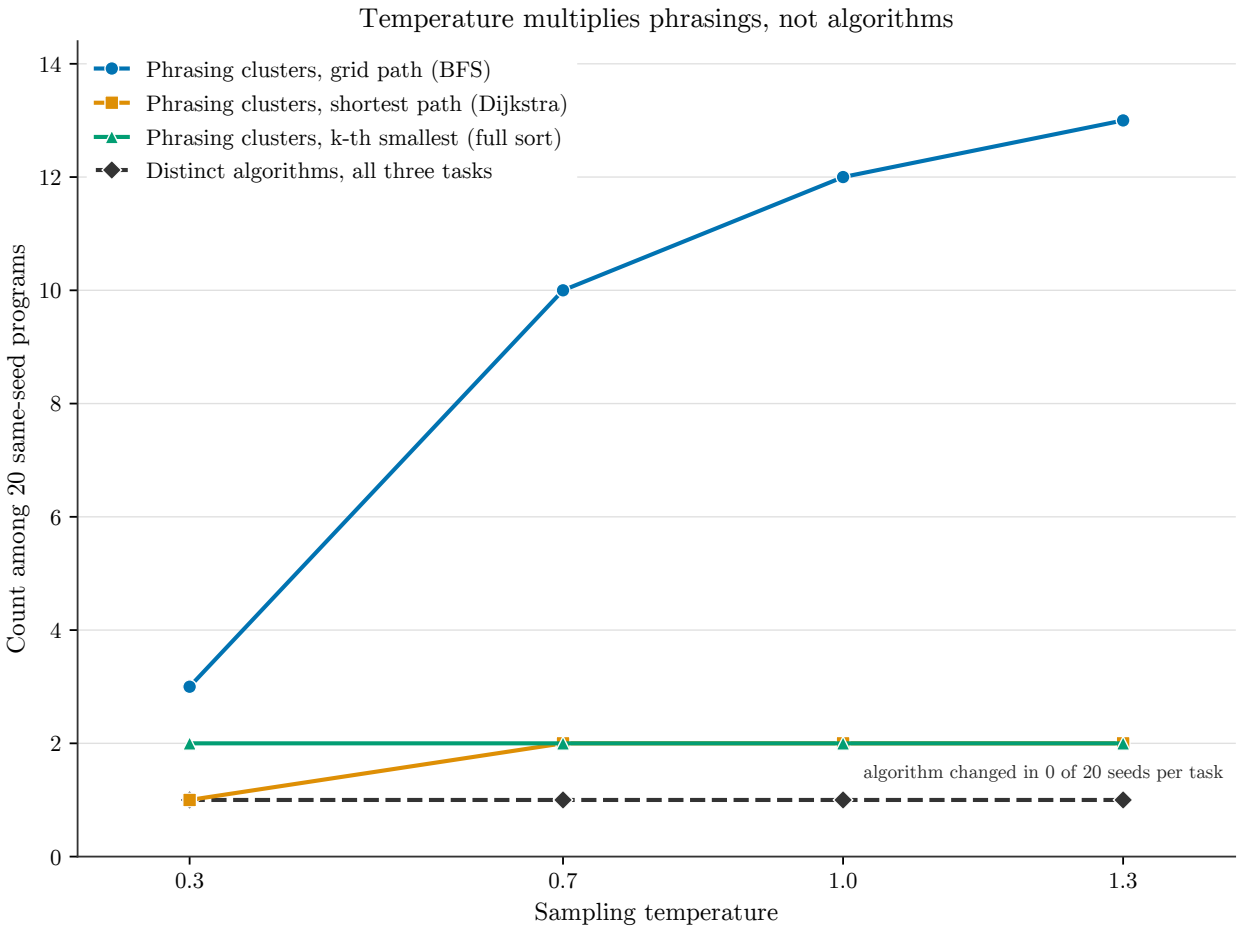


Figure 3: Counts of phrasing clusters and of distinct algorithms across sampling temperature, under a shared random stream (Qwen3-8B, non-thinking mode, top-p 1, 20 seeds per task, one program per seed and temperature; same seeds as Figure 1 and Table 1). Solid lines with markers give phrasing clusters among the 20 programs drawn on the same seeds on each task – blue circles for grid path (BFS), orange squares for shortest path (Dijkstra), green triangles for k-th smallest (full sort) – where a cluster is a connected component of programs whose AST 4-gram Jaccard similarity, with identifiers normalised, is at least 0.8. The black dashed line with diamond markers gives distinct algorithms among the same programs, which is 1 on all three tasks at every temperature, as the annotation inside the figure states (algorithm changed in 0 of 20 seeds per task; exact 95% upper bound 0.168 on the change rate per seed). Phrasing diversity rises steeply with temperature on the grid path task (3 to 13 clusters) and stays close to flat on the other two (1 rising to 2 on shortest path; constant at 2 on k-th smallest), while the dashed line never leaves 1.

change-point rule recovered the same tokens on fresh paths, where the remaining entropy was at most 0.004 bits on the five Qwen3-8B and Gemma units (Table 2). The same structure appears in other tasks and families. Connected components in Qwen3-8B is settled at generated tokens 10 to 11, where a `␣parent` or `def find` line leads to union find and `def dfs` to graph search, with 0 bits left at token 11.[12] Gemma's grid prompt forks at response index 32: `␣visited` leads to DFS ($P(\text{BFS}) = 0.011$), and a newline, `if` or `␣queue` to BFS (0.95 to 1.00). Shortest path has no single forking token within its first 60 tokens.

On the default tasks the forking token is decided by a wide margin. At the located forking position, the teacher-forced fork margin of the default is 7.46 nats on the grid task, 4.85 on shortest path, 17.04 on connected components and 6.10 on k-th smallest. Averaged over five prompt templates instead, the margins on the first three tasks are 6.33, 5.61 and 15.96 nats (Figure 5). Dividing these logits by 1.3 predicts at most 0.02 of samples off the default at $T = 1.3$, and 100 samples at $T = 1.3$ from the forking position produced no program off the default on any of the four tasks. A further 20 samples gave one program off the default on each of the grid and shortest path tasks. The temperature null of Section 5.1 is what these margins predict: a temperature of 1.3 scales a 5 to 17 nat gap by $1/1.3$ and leaves the alternative out of reach.

5.3 The forking-token logits predict the temperature response

Where the forking token is contested, the composed law predicts how temperature moves the algorithm mix (Table 2 and Figure 4). The law was committed before any temperature sample and scored on 100 fresh shared streams per cell. It passed its unit criterion on five of six units: the three development units in Qwen3-8B (MAE 0.029, 0.054 and 0.043) and both held-out Gemma units (0.033 on substring search and 0.014 on first and last position). On first and last position the first forking token is impure, and the second level found inside it lowers the error from 0.051 for the single fork law to 0.014 (bootstrap 95% CI of the difference -0.051 to -0.011). Lowering temperature from 1.0 to 0.3 removed the minority in one direction only on both Gemma units (10:0 and 13:0 discordant streams, McNemar $p = 0.001$ and 1.2×10^{-4}) and on minimum spanning tree weight (18:0, $p = 3.8 \times 10^{-6}$).

Table 2: The composed temperature law against its four rivals: mean absolute error (mean total variation distance over T = 0.3, 0.7, 1.0, 1.3; 100 shared streams per cell). Development units were used to build the law; held-out units were committed and swept before any sweep of a development unit. Lowest error per row in bold. The built-in index, that is, a call to the language's own string search instead of an implemented algorithm, is the minority algorithm on substring search.

Unit (model)	Held out	Minority algorithm	Forking token(s)	Composed law	Flat	Share tempering	Single fork	Highest entropy position	$H(A \mid \text{class})$, bits	Pass
Trapped rain water (Qwen3-8B)	no	Two pointers	18 and 19	**0.029**	0.055	0.070	0.038	0.037	< 0.001	yes
Valid anagram (Qwen3-8B)	no	Fixed count array	24 and 26	0.054	**0.030**	0.091	0.054	0.071	< 0.001	yes
Minimum spanning tree weight (Qwen3-8B)	no	Prim	61	**0.043**	0.093	0.058	**0.043**	0.045	0.004	yes
Substring search (Gemma-3-4B)	yes	Built-in index	31	0.033	0.043	**0.023**	0.033	0.036	0.001	yes
First and last position (Gemma-3-4B)	yes	Binary search	12 and 24	**0.014**	0.073	0.018	0.051	0.017	0.001	yes
Grid path (Llama-3.1-8B)	yes	DFS	after a 52-token opening	0.142	0.138	0.143	0.142	**0.134**	0.63	no

[12]Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-7/experiment-34`

The preregistered claim is graded partial. Five of six scored units pass (0.83, above the 0.70 bar), but only three held-out units were scored where the claim required four, so the held-out criterion is underpowered rather than met. No held-out unit exceeded the disconfirming error of 0.2. The law is also not clearly better than simpler rivals. It has the lowest error on three of six units and beats the flat law on four of six (exact sign test $p = 0.34$). Bootstrap intervals of the difference, resampling streams, exclude 0 on minimum spanning tree weight [-0.050, -0.025], first and last position [-0.081, -0.018] and substring search [-0.041, -0.002]. Share tempering beats it on substring search, and the flat law beats it on valid anagram, where temperature barely moves the mix. The law adds most where an impure first fork needs the second level and where the minority is not simply sharpened away.

On the grid task of Llama-3.1-8B every predictor fails. Llama reaches the forking tokens through many openings, so the two confirmed openings cover about 2% of samples and the committed law is almost the flat law. At $T = 1.3$, 72 of 100 programs run to the token cap and the pass rate falls to 0.04, a collapse into unusable code that no law over algorithm shares predicts. An earlier, smaller test of the single fork law on 50 streams per cell passed on one of three Qwen3-8B units. Replaying each stream's own uniforms through the committed token classes reproduced the observed shares there (0.08, 0.18, 0.18 and 0.16 against 0.10, 0.18, 0.18 and 0.20 on trapped rain water), locating the misses in the finite sample rather than in the mechanism.

Applying temperature only at the forking token tests the account causally. With the dose committed from teacher-forced logits, predicted algorithm shares matched the observed ones with MAE 0.010 to 0.047 on five located units, and Hill-1 among correct programs on valid anagram rose from 1.78 to 2.78 at a cost of 2 percentage points of pass rate.[13] On connected components, a default task, the same treatment reached DFS in only 3 and 9 of 50 programs, and the larger dose cost 14 percentage points of pass rate because the alternative tokens at the fork break the format.

5.4 What sets the forking token

The task specification states carry the choice into the forking token (Table 3). Writing the task specification states of a prompt that names the minority algorithm into its padding twin produced that algorithm in 30 of 30 shared streams on all three mixed units, with every program passing. The padding twin alone gave 0 to 7 of 30, and a random exchange of matched norm 0 to 12 of 30. The argmax at the forking token flips with it (`␣Kr` to `␣Prim`), and the fork margin moves by 4.87 to 22.12 nats toward the minority, against at most 2.00 nats for the random exchange. The same exchange carries a prohibition. We wrote the task specification states of a shortest path prompt that bans Dijkstra, at all 36 blocks, into a neutral twin matched in token length, the same prohibition sentence with irrelevant nouns in place of the algorithm. The twin then wrote correct Bellman-Ford/SPFA in 21 of 30 streams; a post hoc McNemar test against a matched random exchange gives 21:0 ($p = 9.5 \times 10^{-7}$).[14]

Post-training made the fork margins (Figure 5). On identical token ids formatted with the chat template, the fork margin of Qwen3-8B exceeds that of Qwen3-8B-Base by 3.27 nats on the grid task, 13.41 on connected components and 2.11 on shortest path, the three default tasks, and by 1.84 and 1.53 nats on the two mixed units. Without chat tokens the gains on the default tasks are 5.31, 11.05 and 2.69 nats, so the template does not produce them. At 64 ordinary phrasing positions per program the two models differ little: the next-token entropy of the base model is higher by 0.21 to 0.34 nats across the five tasks, and the token of the instruct model is the top token of the base model at 90.6% to 95.3% of phrasing positions on the default tasks. At the forking token, the choice of the instruct model is the second or third choice of the base model in 20, 17 and 19 of 20 programs. On the default tasks the Jensen–Shannon divergence between the two

[13] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-8/experiment-39`

[14] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-6/experiment-30`

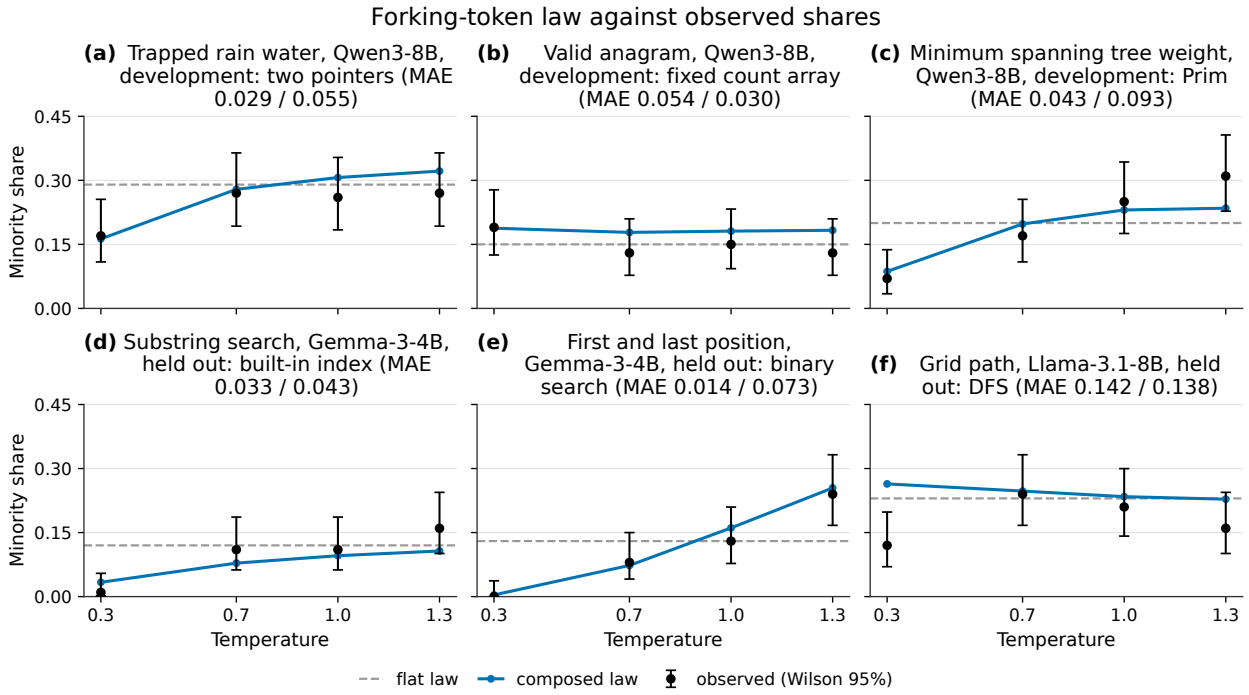


Figure 4: Predicted and observed share of the minority algorithm at four sampling temperatures (0.3, 0.7, 1.0, 1.3; top-p = 1, N = 100 generations per cell, shared random streams across temperatures), for six algorithm-choice units: three Qwen3-8B development units – (a) trapped rain water: two pointers, (b) valid anagram: fixed count array, (c) minimum spanning tree weight: Prim – and three held-out units – (d) Gemma-3-4B substring search: built-in index, that is, a call to the language's own string search, (e) Gemma-3-4B first/last position: binary search, (f) Llama-3.1-8B grid path: DFS. Black circles give the observed minority share with vertical Wilson 95% confidence intervals; the solid blue line with small markers is the composed forking-token law's prediction, fixed before any temperature sample; the dashed grey line is the flat law, a reference that holds the T = 1 share constant across temperature. Each panel title gives the mean absolute error, over the four temperature points, of the composed law and the flat law respectively. The composed law tracks the observed trend, rising or nearly flat, on five of six panels, including both held-out Gemma-3-4B units (d, e); on the held-out Llama-3.1-8B grid path unit (f) both the composed and flat lines stay nearly constant and neither tracks the nonmonotonic observed points, consistent with that unit's pass rate falling from 0.22 to 0.04 between T = 0.3 and T = 1.3, where most generated programs become unusable.

Table 3: Exchanging the task specification states of a forced minority prompt into its padding twin (Qwen3-8B, bfloat16, 30 shared streams per arm). Shares are of attempted programs; the change in fork margin is in nats toward the minority.

Unit	Padding twin	Forced prompt's states exchanged in	Random exchange, matched norm	Exchange vs twin, discordant pairs (p)	Change in fork margin, exchange / random
Minimum spanning tree weight (Prim)	0.23	1.00	0.40	23:0 (1.2×10^{-7})	-4.87 / -1.25
Trapped rain water (two pointers)	0.13	1.00	0.13	26:0 (1.5×10^{-8})	-5.50 / -2.00
Valid anagram (fixed count array)	0.00	1.00	0.00	30:0 (9.3×10^{-10})	-22.12 / $+0.26$

models is 7.3 to 12.0 times larger at the fork than at phrasing positions, but only 1.5 to 1.9 times larger than at phrasing positions matched for base-model entropy, which matches the known concentration of post-training shifts at high-entropy tokens [24]. The instruct default is therefore not the base model's average answer: on minimum spanning tree weight the base model's fork distribution, pushed through the instruct model's continuations, would give Prim 0.68 where the instruct model writes Kruskal 0.78.

Reading the choice before the forking token was not possible with the readouts we tried, which fits a decision made at that token. A natural language autoencoder on Qwen2.5-7B-Instruct [44] named the planned algorithm at the positions that end the prompt in 0 of 112 cases, although at the token that names the algorithm its explanations named it 80% to 89% of the time.[15] Sparse autoencoder features from Gemma Scope 2, the successor release of Gemma Scope [18], read the log odds at the fork of Gemma across 120 prompt variants ($R^2 = 0.66$ against 0.18 for a text baseline), but a ridge readout of the raw residual reached 0.72. Clamping features also steered the fork less than a difference-in-means direction ($+1.78$ against $+4.04$ nats at matched norm). Distributed alignment search at the forking token produced edits that moved the algorithm, but controls with permuted labels and with randomly initialised networks kept 0.63 to 0.87 and 0.69 of the effect, so no certified variable of low rank for the algorithm was found.[16] Path patching located a stack of MLPs and attention heads at the forking position that carries the fork logit, but at the committed dose five matched random components flipped nearly as many programs as the top five (7:1 discordant streams on minimum spanning tree weight, two sided $p = 0.070$, and 0:1 on connected components).[17] A logit-lens reading found no layer at which a ban vetoes the default at the token that first names the algorithm, because the choice is already made before it.[18] A community Jacobian lens failed its own gate against the logit lens on Qwen3-8B (top-1 agreement 0.104 against 0.291 at layer 27), so it was not used further.[19] Thought-anchor resampling of thinking traces [5] under a BFS ban showed DFS fixed from the first resampling point in 4 of 4 traces.

5.5 Other controls reweight the same repertoire

Controls other than temperature moved programs among known algorithms when they moved them at all (Table 4). In Table 4 a fixed push direction *escapes* when, over its first 10 samples, the share of attempts off the default exceeds that of the unpushed prompt by at least 0.20 at a pass rate of at least 0.8 times the unpushed prompt's, with a confirmation at 30 samples by an exact binomial test. Random context seeds left the default intact: with `seed=d32qxk4nt43` or `seed=(horse chair roof love)` prepended, the grid task stayed BFS in 100 of 100 programs and shortest path Dijkstra in 100 of 100, although the word seed moved the residual state at the last prompt token 1.555 times as far as switching between paraphrases of the prompt does. Context with meaning did move the algorithm: a paragraph that implies a relaxation over all edges without naming it turned 61 of 92 correct shortest path programs into Bellman-Ford/SPFA (permutation $p < 0.001$). Instructions moved claims more than code. `Make a fully novel algorithm never seen before` lowered the Dijkstra share by 14.6 points, toward Bellman-Ford/SPFA. Under two 2,000-word novelty instructions, 84% of outputs claimed a new algorithm, 39% of those contained no code, and

[15] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-6/experiment-32

[16] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-8/experiment-40

[17] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-8/experiment-41

[18] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-2/experiment-8

[19] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-2/experiment-12

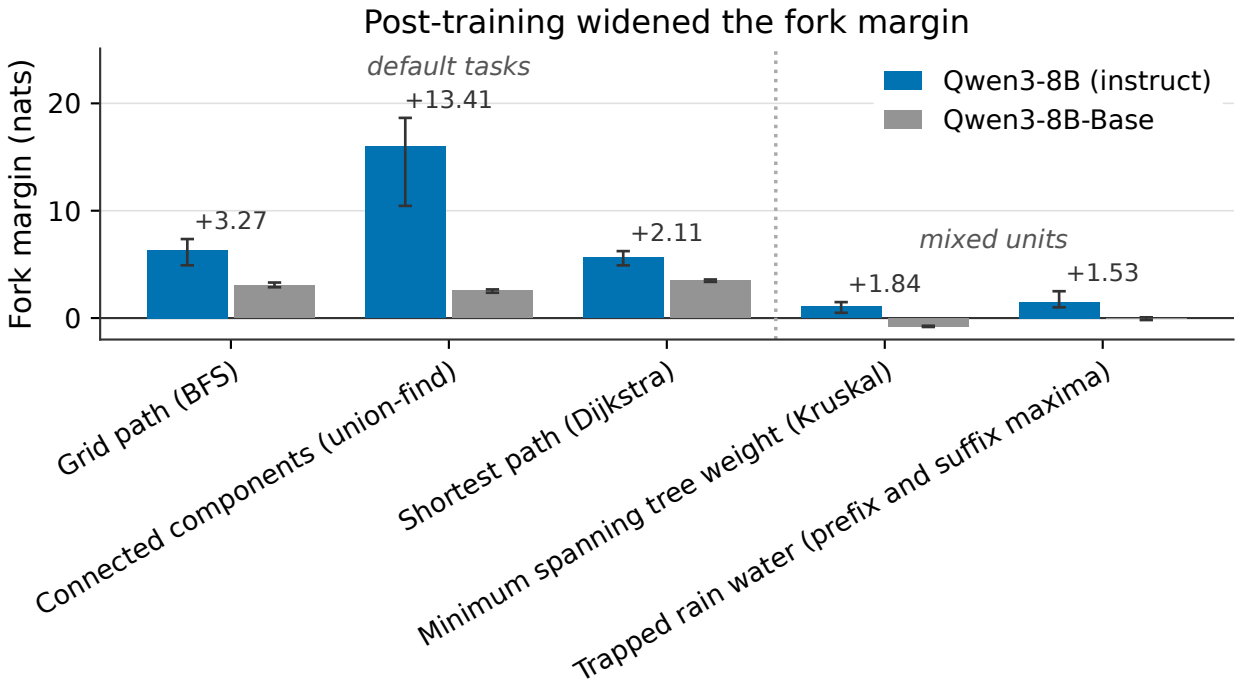


Figure 5: Fork margin — the teacher-forced gap in log probability, in nats, between the token class leading to the default algorithm of the instruct model and the best alternative class — for Qwen3-8B (instruct; dark blue bars) and Qwen3-8B-Base (grey bars) on identical token ids formatted with the chat template, across five coding tasks. Bars show the mean over five prompt templates; whiskers span the minimum to the maximum over those templates. A dotted vertical line and italic headers separate the three default tasks (grid path/BFS, connected components/union find, shortest path/Dijkstra) from the two mixed units (minimum spanning tree weight/Kruskal, trapped rain water/prefix and suffix maxima); the number above each pair is the gain from post-training in mean fork margin, instruct minus base: +3.27, +13.41 and +2.11 nats on the default tasks, +1.84 and +1.53 nats on the mixed units. Over the same five templates and the same programs, the next-token entropy of the base model at 64 ordinary phrasing positions per program, away from the fork, was only 0.21-0.34 nats higher than that of the instruct model, so the effect of post-training on the output distribution is concentrated at the forking token rather than spread across the prompt.

75% of the rest were catalogue algorithms.[20] On online bin packing, the shorter instruction `Make a new algorithm` still produced the textbook best-fit rule in 28 of 29 programs from Qwen3-14B and 28 of 28 from Qwen3-32B, both in thinking mode.[21] An audit of every correct program outside the catalogue from these early decoding conditions found none novel. Asking the model to list ten approaches and implementing one picked at random raised the number of ideas at depth 25 from 1.26 to 3.98 on the grid task and from 1.00 to 3.00 on shortest path, at pass rates of 0.55 and 0.75.

Table 4: What other controls did to the algorithm, and the novelty audit of their correct programs outside the catalogue.

Control	Effect on the algorithm	Correct novel algorithms
Random context seeds (entropy string, four unrelated words)	Grid BFS 100/100, shortest path Dijkstra 100/100	0
Semantic seed (paragraph implying an algorithm)	Shortest path: Bellman-Ford/SPFA 61 of 92 correct ($p < 0.001$); grid: +2.2 points bidirectional BFS ($p = 0.078$)	0
`Make a fully novel algorithm never seen before`	Shortest path: Dijkstra share -14.6 points (to Bellman-Ford/SPFA); grid: -3.7 points	0
2,000-word novelty instructions (575 outputs)	84% claim a new algorithm; 75% of claims with code are catalogue algorithms	0 of 38 correct, outside the catalogue
Ban of the default by name	Lands on one substitute: grid DFS 257 of 257, shortest path Bellman-Ford/SPFA 255 of 260	0
Ban of five grid algorithms by name (10 open configurations)	Only Qwen3-32B with thinking reaches 0.20 correct survivors (17/50 = 0.34 [0.22, 0.48]); others reimplement banned algorithms	0 of 27 audited
Random push of the task specification states (40 fixed directions, $\alpha = 2$, bfloat16)	26 of 40 directions escape on connected components and 6 of 40 on shortest path; landings DFS/BFS and Bellman-Ford/SPFA	0 of 10,651 correct (8-bit engine, 15 tasks)
Temperature at the forking token only	Valid anagram Hill-1 1.78 to 2.78 (pass rate -2 points); connected components DFS at most 9/50	0 of 5,543 correct
Island-model program search (k-th smallest, bin packing, TSP)	k-th smallest: 78 of 2,500 known techniques outside the catalogue vs 2 of 500 for the instruction alone; bin packing: beats best fit on held-out data	8 in one bin packing run (manual read); 0 of 40 in its replication

Bans relocate the default within the repertoire. Whether a single ban is obeyed depends on its wording. On 12 bench tasks the template alone gives a likelihood ratio of 374 on 4 degrees of freedom for avoidance, and the wording `X is not allowed here.` fails on 10 of 12 tasks, in line with the sensitivity of models to prompt formatting [35].[22] Where it is obeyed, a single ban of the default by name lands on one substitute, DFS for BFS and Bellman-Ford/SPFA for Dijkstra, recomputed over all pooled ban samples.[23] Banning five grid algorithms at once fails in most models. Across ten open configurations only Qwen3-32B

[20] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-3/experiment-18`

[21] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-10/experiment-54`

[22] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-3/experiment-13`

[23] Code: `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-3/experiment-16`

with thinking clears the preregistered bar of 0.20 correct survivors.[24] Qwen3-14B with thinking reaches 0.14 and Qwen3-32B without thinking 0.08, while Qwen3-8B with and without thinking, Llama-3.1-8B-Instruct, Qwen3-14B without thinking, gemma-3-27b-it, Qwen2.5-Coder-32B-Instruct and gemma-3-4b-it reach 0.00; these models keep writing BFS or DFS, often without naming it. Lexical compliance, the share of programs that name no banned algorithm, exceeded mechanistic compliance, the share whose runtime behaviour is not a banned algorithm, by 0.70 pooled over ten configurations, a descriptive figure because the expansion-order tracer failed its gold gate.[25] With an exact behavioural instrument on online bin packing, Qwen3-8B without thinking obeyed a ban of five names lexically in 0.91 of programs but by mechanism in 0.51 (difference 0.40 [0.21, 0.55]), while Qwen3-14B and Qwen3-32B with thinking left the banned rules.[26]

The audits found no correct novel algorithm under any control applied to single samples, including 0 of 10,651 correct programs under random pushes and 0 of 4,703 in the sweeps that tested the temperature law (rule-of-three bound 0.0006).[27] Program search did better than the textbook rule without producing a confirmed new idea. On k-th smallest it produced 78 known techniques outside the catalogue in 2,500 calls, against 2 in 500 for the instruction alone (a ratio per call of 7.8, lower 95% bound 2.41, short of the committed tenfold) and 0 in 1,250 for a prompt that enumerates approaches and picks one, all of them published methods.[28] On online bin packing the best searched programs reached 1.40% to 2.43% excess bins over the lower bound on held-out Weibull instances against 3.98% for best fit, but none beat the published FunSearch heuristic (0.68%).[29] In an independent replication with two new seeds, the arms that forced a token or asked for a different idea beat best fit by the preregistered margin, an excess at most 0.8 times best fit's (1.72% to 2.40% on the Weibull set), and the two temperature arms did not (3.67%, and best fit's own 3.98%). In the first search only one of three judge families passed its control gate, so under the committed rule a manual read, made without knowing which arm wrote each program, decided novelty; it classed eight programs as a new idea, all preferring a leftover that is a whole multiple of the item size, and that one judge agreed on all eight; the replication found 0 novel among 40 judged programs.[30] The same searches tested where to spend randomness, at matched numbers of calls. Forcing a token of rank 2 to 8 at the position of highest entropy among the first 48 code tokens gave 3.0 and 17.8 times the behaviour clusters per call of temperature 1.0 in the two bin packing searches, but 2.10 and 0.74 times those of the same forcing at a random position, and on constructive TSP it gave 0.96 times the clusters of temperature 1.0, which met the preregistered disconfirming condition.[31]

5.6 Where a ban of five names lands in activation space

The one configuration that obeys the ban of five names moves to other known algorithms, among them unit weight Dijkstra (6 of its 17 correct survivors) and Bellman-Ford/SPFA (7), and whether unit weight

[24] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-8/experiment-44

[25] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-9/experiment-45

[26] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-10/experiment-54

[27] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-4/experiment-22

[28] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-7/experiment-37

[29] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-10/experiment-51

[30] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-10/experiment-52

[31] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-10/experiment-53

Dijkstra lies at the edge of the banned algorithms or opposite them depends on how the map is built (Figure 6). Qwen3-32B with thinking writes BFS in 91 of 100 baseline programs; under the ban its six correct catalogue survivors are all unit weight Dijkstra, and on held-out ban templates 9 of 9 are. On a map of the ten grid algorithms built from forced prompts that name them (block 31), unit weight Dijkstra is the unbanned algorithm farthest from the banned five, with a geodesic distance, measured on a k-nearest-neighbour graph of the individual states with $k = 15$, of 1.63 times the median of the unbanned algorithms.[32] On a map built from name-scrubbed programs (block 33), that is, programs with every algorithm name removed, it lies at the edge, at 0.67 times the median and nearest to greedy best-first search. A later experiment that crossed six expansion disciplines with six containers (24 implementable cells, 287 name-scrubbed programs) found that this program map tracks the container more than the expansion discipline (partial Spearman 0.28 against 0.41 on Qwen3-32B and 0.23 against 0.30 on Qwen3-8B), so its edge placement is not yet a placement by mechanism; on that frame the Bellman-Ford/SPFA survivors land beside relaxation programs, and the covert BFS and DFS programs of configurations that disobey the ban land inside the banned classes (0.98 to 0.99).[33] Distances on the map built from names track the lexical overlap of the algorithm names (Spearman -0.61 at $k = 15$) and the relation was not detected within algorithms in a test with aliases of the same algorithm (Spearman -0.12, $p = 0.46$; three algorithms, underpowered), so names and mechanisms are confounded there.[34] The landing evidence is weak at the level of algorithms: the permutation p over algorithms is 0.20, its floor, and the opposite reading held in 10 of 15 robustness cells. The geometry is also correlational. Pushing Qwen3-8B's grid states inside the named map toward the survivor centroids produced BFS in 300 of 300 programs.

6 Discussion

The headline contrast follows from where each control acts. The algorithm a code model writes behaves like an almost discrete decision taken at one or two tokens early in the program, and temperature acts on that decision only through the next-token distribution there; a push of the task specification states acts upstream, on the states that set that distribution (Section 5.4). This single account explains the three regimes we observed. Where post-training has widened the fork margin to 4.85 to 17.04 nats, a temperature of 1.3 cannot reach the alternative, so the algorithm is fixed while every other position, where the next token is less certain, varies more. That is the phrasing growth of Figure 3. Where the forking token is contested, temperature sharpens or flattens the same two or three known algorithms at the rate the forking-token logits predict. And where the model reaches the decision through many openings, as Llama does on the grid task, the choice is spread over the opening and no law at a single token captures it. In the terms of the question that motivates this work, a tight temperature with top-k 10 gives one idea in a few phrasings, and a loose one gives the same idea, or the same few ideas, in more phrasings.

This account reconciles two findings that look contradictory. Lee et al. [17] report that temperature above 1 raises algorithmic diversity among correct programs, and we find that temperature never changes the algorithm on default tasks. Both hold. On tasks whose fork is contested, temperature moves mass between known algorithms at the predicted rate; Prim on minimum spanning tree weight rises from 0.07 at $T = 0.3$ to 0.31 at $T = 1.3$. On tasks whose fork is decided by several nats, it does not. A diversity evaluation that pools tasks will see an average of the two. Our results also bear on the claim that a model returns the average answer and that only context moves it elsewhere. The instruct default is not the base model's average:

[32] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-9/experiment-46

[33] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-10/experiment-55

[34] Code: https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract/tree/fork/run_jSassd-YWLVC/round-9/evaluation-8

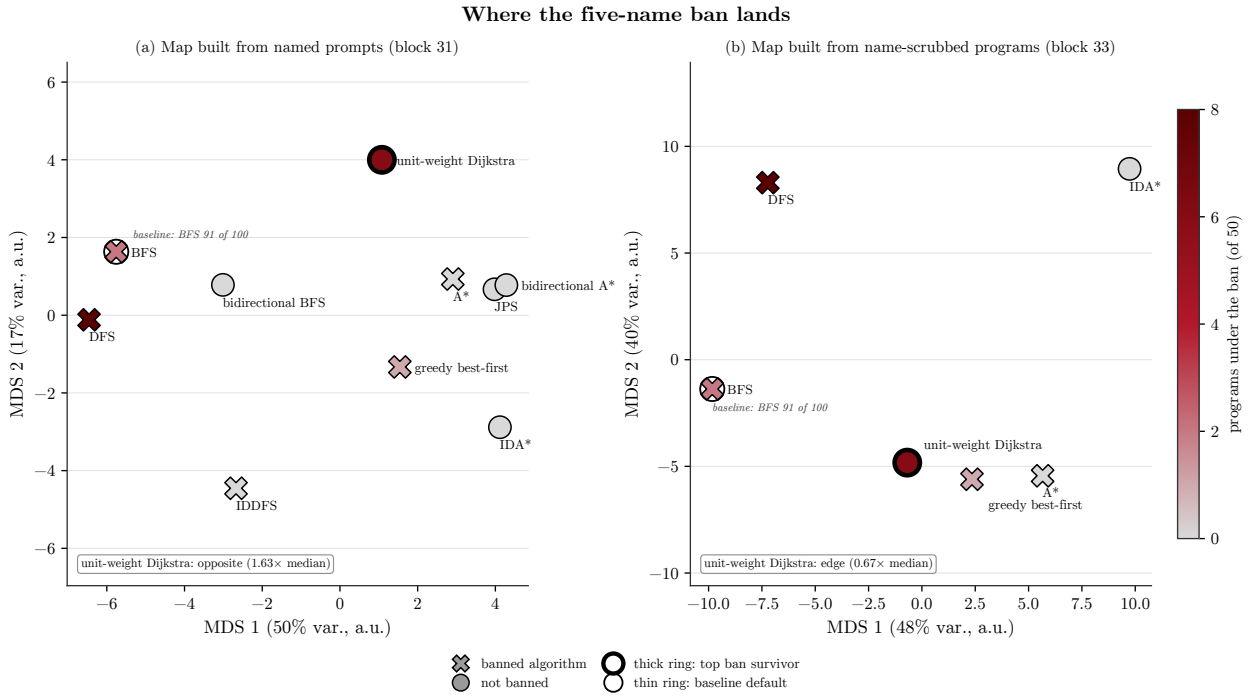


Figure 6: Maps in activation space of ten grid search algorithms in Qwen3-32B (4-bit weights, thinking mode), with point coordinates from classical multidimensional scaling (MDS; arbitrary axis orientation, arbitrary units) of Euclidean distances between the residual-stream centroids of the algorithms. Marker shape distinguishes the five algorithms named in the ban, that is, BFS, DFS, IDDFS, A* and greedy best-first (cross), from the rest (circle). Marker fill colour, on the shared colour bar at right ("programs under the ban (of 50)", light grey to dark red, 0-8), gives how many of 50 programs sampled under the ban of five names implemented that algorithm. A thick black ring marks unit weight Dijkstra, the unbanned catalogue algorithm written most often under the ban (6/50; seven further correct programs used Bellman-Ford/SPFA, which is not on these maps); a thin ring marks BFS, the modal default with no ban in effect (91/100, noted beside it). (a) Map built from named prompts (block 31): centroids of 300 forced `Implement ⟨algorithm⟩` prompts (30 per algorithm, 10 algorithms), averaged over the task specification tokens; the top two MDS axes carry 50% and 17% of the variance. On a k-nearest-neighbour graph with k=15, unit weight Dijkstra sits opposite the banned set (geodesic distance 1.63× the median distance among unbanned algorithms). (b) Map built from name-scrubbed programs (block 33), that is, programs with every algorithm name removed: centroids of such programs that pass the hidden tests, for the six algorithms with at least 10 programs (20 each, 17 for IDA*; 117 programs total), averaged over the generated code tokens; the top two axes carry 48% and 40% of the variance. On the same k=15 construction, unit weight Dijkstra instead sits at the edge of the banned set (0.67× the median). The two maps disagree on whether the ban's main catalogue survivor lands opposite the banned algorithms or beside them; the geometry is correlational, not a steering result.

post-training moved and sharpened one token, often promoting the base model's second or third choice, and left the repertoire unchanged.

For practice, the account says where to spend randomness. Temperature is a poor exploration control for algorithm choice, and on contested units its effect can be predicted before sampling from the forking-token logits. Inside program search, however, targeting the forking token was not reliably better than forcing a token at a random position (Section 5.5), so the account locates the decision without yet telling a search where to spend its randomness. Controls that act upstream of the fork, such as context that implies an algorithm, a prompt that lists approaches and then picks one, or perturbations of the task specification states, do change the algorithm, but they move it to the second choice the model already knows, and at a cost in correctness. The audits found no route from any control applied to single samples to a correct novel algorithm. An evaluator that keeps improvements across generations made the same 8B model better than the textbook rule on bin packing, which is where the field's discovery systems operate [28, 34]. Whether it made the model new remains open: one search's manual read found eight programs with a new idea, and the replication found none.

For interpretability, the results suggest that readouts of a plan taken before the code starts should be validated against the forking token. Natural language and sparse-feature readouts found no verbalised plan before the code, and fitted subspace edits at the fork moved the algorithm without passing the controls that would make them a variable of the model, consistent with the interpretability illusions that subspace patching can produce [22]. The forking token is where intervention is cheap and measurable, but a certified mechanism for the choice is still missing.

7 Limitations

The headline contrast between the push and temperature ran only in an 8-bit (W8A8) engine and was not rerun in bfloat16; the push alone changes the algorithm in bfloat16 at the same absolute push norm, but the paired contrast was not repeated, and it pools two tasks, with the grid task moving only 2.50 of 20 seeds. Most results come from one model, Qwen3-8B, in non-thinking mode. The composed law was tested on three held-out units in two other families; two passed and Llama's grid task failed, below the four held-out units the preregistered claim required, so the law's generality is an open question. The law was also not clearly better than the flat law, a baseline that predicts no change (four of six units, sign test $p = 0.34$), which means that much of its accuracy reflects how little temperature moves these units. The temperature null on shared seeds rests on 20 seeds per task, which bounds the change rate per seed below 0.17 per task, not at zero, and it ran with 8-bit linear layers; the bfloat16 fork margins and the grid sweep with independent samples agree with it, but the thinking-mode evidence is limited to shortest path and a 10-seed grid arm.

The tasks are short, well-known programming problems whose solutions are likely memorised, with catalogues of three to ten known algorithms; the conclusions may not hold for open-ended programs or for problems without a textbook solution. Experiments ran on different engines (bfloat16 and 8-bit linear layers, vLLM and Hugging Face); push escapes reproduced in bfloat16, but pushed cells showed precision sensitivity at the level of single streams. Phrasing clusters depend on a Jaccard threshold over AST shingles with normalised identifiers, and the growth was driven mostly by the grid task.

Labels and the novelty audit carry their own uncertainty. Labellers were gated per task, but the gate failed by its strict criteria for the decoding experiment with independent samples and for the agreement between runtime and AST labels on the path tasks ($\kappa = 0.753$), and the expansion-order tracer used for the ban of five names failed its gold gate, so those numbers are descriptive. In the program searches at most one LLM judge family passed its control gate, short of the two the bin packing rule required, so novelty there rests on manual reads, and the eight candidates of one search's reader, which the one gated judge also called new, were not reproduced by the replication. The geometry in activation space is correlational, and the map built

from names is confounded by lexical overlap between algorithm names.

8 Conclusion

On the same random streams, a random push of Qwen3-8B's task specification states changed the algorithm in more seeds than temperature did, by 0.545 per seed (95% CI 0.41 to 0.67) over shortest path and k-th smallest, a preregistered contrast that held in an 8-bit engine and is untested in bfloat16. Temperature from 0.3 to 1.3 left the default algorithm unchanged in all 60 seeds (20 per task) while phrasings multiplied, because a forking token, not temperature, chooses the algorithm. Fork margins of 4.85 to 17.04 nats on the default tasks, widened by post-training, account for the null, and on contested forks the forking-token logits committed before sampling predicted the temperature response within 0.014 to 0.054 total variation on five of six units. No control applied to single samples produced a correct algorithm outside the known repertoire; with an evaluator selecting across generations, one bin packing search yielded eight programs classed as a new idea, and its replication found none.

Future work follows directly:

- testing the composed law on more held-out units and families, including models that reach the fork through many openings;
- extending the forking-token analysis to thinking traces, where the first approach named was fixed in every trace we read;
- testing, with more seeds and tasks, whether forcing branches at the forking token adds anything over forcing them at a random position inside program search, which our two bin packing searches and one TSP search left unsettled;
- searching for a variable of the algorithm choice that passes controls with permuted labels and with randomly initialised networks.

Code and Data Availability

Code, prompts, raw generations, algorithm labels and the preregistration hash chains of every experiment are public at `https://github.com/ai-inventor-papers/ai-invention-295f8e-prompt-bans-substitute-not-just-subtract`; the footnotes in Sections 1 to 5 link each result to the code that produced it.

References

[1] Eltayeb Ahmed, Uljad Berdica, Martha Elliott, Danijela Horak, and Jakob N. Foerster. Intent Factored Generation: Unleashing the diversity in your language model. *arXiv preprint arXiv:2506.09659*, 2025. doi: 10.48550/arXiv.2506.09659.

[2] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems*, 2024. doi: 10.48550/arXiv.2406.11717.

[3] Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023. doi: 10.48550/arXiv.2303.08112.

[4] Eric J. Bigelow, Ari Holtzman, Hidenori Tanaka, and Tomer D. Ullman. Forking paths in neural text generation. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2412.07961.

[5] Paul C. Bogdan, Uzay Macar, Neel Nanda, and Arthur Conmy. Thought Anchors: Which LLM reasoning steps matter? *arXiv preprint arXiv:2506.19143*, 2025. doi: 10.48550/arXiv.2506.19143.

[6] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/BIOMET/26.4.404.

[7] Atticus Geiger, Zhengxuan Wu, Christopher Potts, Thomas Icard, and Noah D. Goodman. Finding alignments between interpretable causal variables and distributed neural representations. In *Conference on Causal Learning and Reasoning (CLeaR)*, 2024. doi: 10.48550/arXiv.2303.02536.

[8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[9] M. O. Hill. Diversity and evenness: A unifying notation and its consequences. *Ecology*, 54(2):427–432, 1973. doi: 10.2307/1934352.

[10] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

[11] Audrey Huang, Adam Block, Dylan J. Foster, Dhruv Rohatgi, Cyril Zhang, Max Simchowitz, Jordan T. Ash, and Akshay Krishnamurthy. Self-improvement in language models: The sharpening mechanism. *arXiv preprint arXiv:2412.01951*, 2024. doi: 10.48550/arXiv.2412.01951.

[12] Ly Tran Ho Khanh, Dongxuan Zhu, Man-Chung Yue, and Viet Anh Nguyen. Test-time diverse reasoning by Riemannian activation steering. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40 (37):31429–31437, 2026. doi: 10.1609/aaai.v40i37.40407.

[13] Heejun Kim, Seungpil Lee, Jewon Yeom, Jaewon Sok, Seonghyeon Park, Jeongjae Park, Taesup Kim, and Sundong Kim. From noise to diversity: Random embedding injection in LLM reasoning. *arXiv preprint arXiv:2605.11936*, 2026. doi: 10.48550/arXiv.2605.11936.

[14] Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2310.06452.

[15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

[16] Sangmook Lee, Minbeom Kim, Jeonghye Kim, Dohyung Kim, Sojeong Rhee, and Kyomin Jung. Are we measuring strategy or phrasing? The gap between surface- and approach-level diversity in LLM math reasoning. *arXiv preprint arXiv:2606.29985*, 2026. doi: 10.48550/arXiv.2606.29985.

[17] Seonghyeon Lee, HeeJae Chon, Joonwon Jang, Dongha Lee, and Hwanjo Yu. How diversely can language models solve problems? Exploring the algorithmic diversity of model-generated code. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 152–167, 2025. doi: 10.18653/v1/2025.findings-emnlp.10.

[18] Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca D. Dragan, Rohin Shah, and Neel Nanda. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, 2024. doi: 10.18653/v1/2024.blackboxnlp-1.19.

[19] Bill Yuchen Lin, Abhilasha Ravichander, Ximing Lu, Nouha Dziri, Melanie Sclar, Khyathi Raghavi Chandu, Chandra Bhagavatula, and Yejin Choi. The unlocking spell on base LLMs: Rethinking alignment via in-context learning. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2312.01552.

[20] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pages 32201–32223, 2024.

[21] Yining Lu, Dixuan Wang, Tianjian Li, Dongwei Jiang, Sanjeev Khudanpur, Meng Jiang, and Daniel Khashabi. Benchmarking language model creativity: A case study on code generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2776–2794, 2025. doi: 10.18653/v1/2025.naacl-long.141.

[22] Aleksandar Makelov, Georg Lange, Atticus Geiger, and Neel Nanda. Is this the subspace you are looking for? An interpretability illusion for subspace activation patching. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2311.17030.

[23] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

[24] Haoming Meng, Kexin Huang, Shaohang Wei, Chiyu Ma, Shuo Yang, Xue Wang, Guoyin Wang, Bolin Ding, and Jingren Zhou. Sparse but critical: A token-level analysis of distributional shifts in RLVR fine-tuning of LLMs. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2603.22446.

[25] Sonia Krishna Murthy, Tomer Ullman, and Jennifer Hu. One fish, two fish, but not the whole sea: Alignment reduces language models' conceptual diversity. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11241–11258, 2025. doi: 10.18653/v1/2025.naacl-long.561.

[26] Akanksha Narula, Mofasshara Binte Rafique, and Laurent Bindschaedler. The Invisible Lottery: How subtle cues steer algorithm choice in LLM code generation. *arXiv preprint arXiv:2606.04057*, 2026. doi: 10.48550/arXiv.2606.04057.

[27] Minh Nhat Nguyen, Andrew Baker, Clement Neo, Allen Roush, Andreas Kirsch, and Ravid Shwartz-Ziv. Turning up the heat: Min-p sampling for creative and coherent LLM outputs. In *International Conference on Learning Representations*, 2025.

[28] Alexander Novikov, Ngân Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. doi: 10.48550/arXiv.2506.13131.

[29] Theo X. Olausson, Metod Jazbec, Xi Wang, Armando Solar-Lezama, Christian A. Naesseth, Stephan Mandt, and Eric Nalisnick. A tale of two temperatures: Simple, efficient, and diverse sampling from diffusion language models. In *Conference on Language Modeling (COLM)*, 2026. doi: 10.48550/arXiv .2604.09921.

[30] Max Peeperkorn, Tom Kouwenhoven, Dan Brown, and Anna Jordanous. Is temperature the creativity parameter of large language models? In *Proceedings of the 15th International Conference on Computational Creativity (ICCC)*, 2024. doi: 10.48550/arXiv.2405.00492.

[31] Xiangyu Qi, Ashwinee Panda, Kaifeng Lyu, Xiao Ma, Subhrajit Roy, Ahmad Beirami, Prateek Mittal, and Peter Henderson. Safety alignment should be made more than just a few tokens deep. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2406.05946.

[32] Shauli Ravfogel, Anej Svete, Vésteinn Snæbjarnarson, and Ryan Cotterell. Gumbel counterfactual generation from language models. In *International Conference on Learning Representations*, 2025.

[33] Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522, 2024. doi: 10.18653/v1/2024.acl-long.828.

[34] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, et al. Mathematical discoveries from program search with large language models. *Nature*, 625 (7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[35] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2310.11324.

[36] Shairoz Sohail and Taher Ali. Towards a measure of algorithm similarity. *arXiv preprint arXiv:2510.27063*, 2025. doi: 10.48550/arXiv.2510.27063.

[37] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000. doi: 10.1126/SCIENCE.290.5500.2319.

[38] Evan Z. Wang, Federico Cassano, Catherine Wu, Yunfeng Bai, Will Song, Vaskar Nath, Ziwen Han, Sean M. Hendryx, Summer Yue, and Hugh Zhang. Planning in natural language improves LLM search for code generation. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2409.03733.

[39] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. In *Advances in Neural Information Processing Systems*, 2025. doi: 10.48550/arXiv.2506.01939.

[40] Peter West and Christopher Potts. Base models beat aligned models at randomness and creativity. *arXiv preprint arXiv:2505.00047*, 2025. doi: 10.48550/arXiv.2505.00047.

[41] Fan Wu, Emily Black, and Varun Chandrasekaran. Generative monoculture in large language models. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2407.02209.

[42] Zhengxuan Wu, Atticus Geiger, Thomas Icard, Christopher Potts, and Noah D. Goodman. Interpretability at scale: Identifying causal mechanisms in Alpaca. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.48550/arXiv.2305.08809.

[43] Daniel Wurgaft, Can Rager, Matthew Kowal, Vasudev Shyam, Sheridan Feucht, et al. Manifold steering reveals the shared geometry of neural network representation and behavior. *arXiv preprint arXiv:2605.05115*, 2026. doi: 10.48550/arXiv.2605.05115.

[44] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115.

[45] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[46] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, 2024. doi: 10.48550/arXiv.2402.01145.

[47] Longfei Yun, Chenyang An, Zilong Wang, Letian Peng, and Jingbo Shang. The price of format: Diversity collapse in LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15454–15468, 2025. doi: 10.18653/v1/2025.findings-emnlp.836.

[48] Cheng Zhang, Rui Xin, and Chudi Zhong. Where you inject diversity matters: A unified framework for diverse generation. *arXiv preprint arXiv:2606.10302*, 2026. doi: 10.48550/arXiv.2606.10302.

[49] Jiayi Zhang, Simon Yu, Derek Chong, Anthony Sicilia, Michael R. Tomz, Christopher D. Manning, and Weiyan Shi. Verbalized Sampling: How to mitigate mode collapse and unlock LLM diversity. *arXiv preprint arXiv:2510.01171*, 2025. doi: 10.48550/arXiv.2510.01171.

[50] Shimao Zhang, Yu Bao, and Shujian Huang. EDT: Improving large language models' generation by entropy-based dynamic temperature sampling. *arXiv preprint arXiv:2403.14541*, 2024. doi: 10.48550/arXiv.2403.14541.

[51] Yiming Zhang, Harshita Diddee, Susan Holm, Hanchen Liu, Xinyue Liu, Vinay Samuel, Barry Wang, and Daphne Ippolito. NoveltyBench: Evaluating language models for humanlike diversity. *arXiv preprint arXiv:2504.05228*, 2025. doi: 10.48550/arXiv.2504.05228.

[52] Dongxuan Zhu, Ly Tran Ho Khanh, Andy Yat-Ming Cheung, Man-Chung Yue, and Viet Anh Nguyen. Exploring diverse generation paths via inference-time Stiefel activation steering. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2601.22010.

[53] Yuqi Zhu, Jia Li, Ge Li, YunFei Zhao, Jia Li, Zhi Jin, and Hong Mei. Hot or cold? Adaptive temperature sampling for code generation with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(1):437–445, 2024. doi: 10.1609/aaai.v38i1.27798.